AC-BaaS: An Asynchronous Cross-Blockchain as a Service for the Internet of Things

Lingxiao Yang, *Member, IEEE*, Xuewen Dong, *Member, IEEE*, Zhiguo Wan, *Senior Member, IEEE*, Sheng Gao, *Member, IEEE*, Wei Tong, *Member, IEEE*, Yong Yu, *Fellow, IEEE*, and Yulong Shen, *Senior Member, IEEE*

***Abstract*—Cross-chain techniques improve blockchain scalability and interoperability, providing decentralized exchange and cross-chain collaboration services for Internet of Things (IoT) data across various domains. However, current state-of-the-art (SOTA) solutions for cross-chain data exchange across multiple domains are constrained by synchronous networks, hindering efficient data exchange in intermittent network environments. Furthermore, there is a lack of research on asynchronous cross-chain transaction pool mechanisms, which are crucial for optimizing system utility. In this paper, we propose AC-BaaS, an asynchronous cross-blockchain as a service framework tailored for the multi-domain IoT. Built upon a specially designed asynchronous sidechain architecture, the system leverages a committee to provide AC-BaaS for data exchange across multiple IoT domains. To fulfill the need for asynchronous and efficient data exchange, we combine the ideas of aggregate signatures and verifiable delay functions to devise a novel cryptographic primitive called delayed aggregate signature (DAS), which constructs asynchronous cross-chain proofs (ACPs) that ensure the security of cross-chain interactions. To ensure the consistency of asynchronous transactions, we propose a multilevel buffered transaction pool that guarantees the transaction sequencing. We further propose a heuristic for optimizing the utility of the buffer pool mechanism to strike a balance between performance and resource consumption. We also examine DAS delay size settings to trade-off security and efficiency. We analyze and prove the security of AC-BaaS, simulate asynchronous communication environments under various security levels, and conduct a comprehensive evaluation. The results show that AC-BaaS outperforms SOTA schemes, improving throughput by an average of 1.71 to 5.09 times, reducing transaction latency by 64.36% to 85.49%, and maintaining comparable resource overhead.**

***Index Terms*—Blockchain, sidechain, multi-domain data exchange, asynchronous cross-chain proofs, Internet of Things.**

Received 2 September 2025; revised 15 November 2025; accepted 19 December 2025. Date of publication 24 December 2025; date of current version 5 February 2026. This work was supproted in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02025, in part by the National Key R&D Program of China under Grant 2023YFB3107500, in part by the National Natural Science Foundation of China under Grant U24B20149, Grant 62272385, Grant U23A20302, and Grant 62220106004, in part by the Key Research and Development Program of Shaanxi under Grant 2024GX-ZDCYL-01-09, and in part by the Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research under Grant ZR2022ZD03. An earlier version of this article was presented at the IEEE International Conference on Computer Communications (INFOCOM) [DOI: 10.1109/INFOCOM55648.2025.11044780]. *(Corresponding authors: Xuewen Dong; Yong Yu.)*

Lingxiao Yang and Yong Yu are with the School of Artificial Intelligence and Computer Science, Shaanxi Normal University, Xi'an 710119, China (e-mail: lxyang@snnu.edu.cn; yuyong@snnu.edu.cn).

Xuewen Dong is with the School of Computer Science and Technology, Xidian University, Xi'an 710071, China, also with the Engineering Research Center of Blockchain Technology Application and Evaluation, Ministry of Education, Xidian University, Xi'an 710071, China, and also with the Shaanxi Key Laboratory of Blockchain and Secure Computing, Xi'an 710126, China (e-mail: xwdong@xidian.edu.cn).

Zhiguo Wan is with Zhejiang Lab, Hangzhou 311121, China (e-mail: wanzhiguo@zhejianglab.com).

Sheng Gao is with the School of Information, Central University of Finance and Economics, Beijing 100081, China (e-mail: sgao@cufe.edu.cn).

Wei Tong is with the School of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou 310018, China (e-mail: wtong@zstu.edu.cn).

Yulong Shen is with the School of Computer Science and Technology, Xidian University, Xi'an 710071, China, and also with the Shaanxi Key Laboratory of Network and System Security, Xi'an 710126, China (e-mail: ylshen@mail.xidian.edu.cn).

Digital Object Identifier 10.1109/TSC.2025.3647639

I. Introduction

THE Internet of Things (IoT) has been widely applied in scenarios such as smart cities, energy management, and healthcare. The ubiquity of IoT facilitates ubiquitous data interaction and services computing between smart terminals and sensor devices [2]. For example, in mobile crowd-sensing, groups of individuals with mobile devices capable of sensing and computing, e.g., smartphones and wearables, can share data to infer a more comprehensive perception of the environment. In energy trading, sharing IoT data from different energy domains, e.g., solar and wind, among producers, consumers, and intermediaries, can help optimize peer-to-peer (P2P) energy transaction services. According to Cisco, the number of IoT devices worldwide is expected to reach 500 billion by 2030 [3]. As such, equipping IoT with a secure, efficient, and scalable multi-domain data exchange infrastructure is critical. However, existing IoT data storage and sharing infrastructures are mostly centralized services. Organizations responsible for IoT devices in different domains need to rely on a third-party central authority for data aggregation and management, thereby limiting scalability and increasing the risk of a single point of failure [4].

The increasing interest in blockchain technology has driven researchers to explore decentralized solutions to enhance IoT capabilities [5], [6]. However, due to resource limitations of IoT devices and potential privacy concerns, it is difficult to directly apply public blockchains to IoT scenarios. Permissioned blockchains offer data immutability and multi-party maintenance benefits of public blockchains while providing higher performance, flexibility, and privacy protection. Consequently, some solutions explore using permissioned blockchains to meet the dynamic high-performance requirements of IoT scenarios [7], but still require cross-chain interaction capabilities between permissioned blockchains [8].

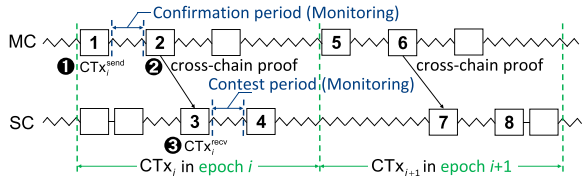


Fig. 1. Illustration for current sidechain-based cross-chain interactions. Wavy lines indicate omitted blocks. Blocks: **1.** $\mathsf{CTx}_i^{\mathsf{send}}$ is initiated; **2.** The cross-chain proof of $\mathsf{CTx}_i^{\mathsf{send}}$ is sent to SC; **3.** A $\mathsf{CTx}_i^{\mathsf{recv}}$ is created; **4.** $\mathsf{CTx}_i^{\mathsf{recv}}$ is stabilized on SC, and then CTx_i is completed; **5 to 8.** The CTx_{i+1} is executed at the next epoch.

Sidechain provides a secure and efficient cross-chain solution with two modes: (i) *Parent-child mainchain-sidechain*. The traditional sidechains are pegged to the mainchain and achieve scaling by offloading traffic from the mainchain. Sidechain nodes monitor the mainchain states, and the mainchain verifies the cross-chain proofs from the sidechain maintainers for cross-chain interactions [9]. (ii) *Parallel sidechain*. Sidechain and mainchain are independent parallel blockchains, which can act as each other's sidechain and verify mutual cross-chain proofs for two-way cross-chain interactions [10], [11]. Considering context, the latter is more suitable for data exchange among organizations in different IoT domains. Each organization runs an independent permissioned blockchain to manage IoT data and cross-chain communication without third-party dependency through cross-chain proofs.

Motivation: (i) Currently, sidechain applications are primarily focused on cryptocurrency exchanges on public blockchains, where security is prioritized over performance in synchronous network environments [9], [10], [11], [12], as shown in Fig. 1. During cross-chain processes, both the mainchain and sidechain must continuously monitor block states, which increases communication overhead (see Appendix A, available in the online for details). This design does not meet the requirements for efficient cross-domain data exchange in intermittent network environments for resource-constrained devices. (ii) In permissioned blockchains, most solutions rely on trusted relays for cross-chain interactions. However, introducing third-party relays increases system complexity and privacy risks [13], [14]. (iii) Recent advances leverage simplified payment verification (SPV), threshold signatures, and zero-knowledge proofs to generate cross-chain proofs [10], [15], [16]. However, to ensure consistency in cross-chain interactions, these methods still depend on strict synchronization between blockchains, resulting in significant on-chain computational overhead. (iv) Furthermore, current research on transaction pools is primarily focused on single-blockchain systems, and research and optimization of asynchronous cross-chain transaction pool mechanisms remain an unexplored area.

In addition, asynchronous cross-chain interaction differs from the synchronous method in that asynchronous cross-chain protocols typically achieve higher performance. This is because it departs from the strict sequential nature of transaction execution in synchronous mode and provides asynchronous concurrency. However, protocol design under asynchronous conditions poses greater challenges and must address how to ensure data consistency and security across different blockchains in the absence of a global clock.

Challenge: Ensuring the persistence and liveness of blockchains for cross-chain interactions without synchronous network settings is a significant challenge for asynchronous cross-chain communication. In other words, it is necessary to ensure that valid asynchronous cross-chain transactions can be smoothly executed, eventually recorded in blocks, and confirmed as stable by a sufficient number of subsequent blocks. This poses the following technical challenges: (i) How to construct a cross-chain interaction mode in asynchronous environments to enable efficient cross-domain data exchange for nodes with resource constraints and unstable network connections. (ii) How to design lightweight and secure asynchronous cross-chain proofs to reduce the computational burden. (iii) How to ensure the sequential consistency of cross-chain transactions in asynchronous mode. (iv) How to optimize the asynchronous cross-chain transaction pool mechanism to achieve a balance in utility.

Building upon the Blockchain as a Service (BaaS) [17] concept in the fields of blockchain and services computing, this paper proposes an IoT-oriented **a**synchronous **c**ross-**b**lockchain **a**s **a** **s**ervice, **AC-BaaS**, which utilizes the designed asynchronous sidechain (AsyncSC) alternative structure [1]. AsyncSC focuses on a parallel sidechain model without the need for synchronous network setup between blockchains. Unlike traditional BaaS, which provides blockchain technology services through cloud service platforms, AC-BaaS offers cross-chain services in asynchronous environments through a trusted internal committee composed of blockchain maintainers, thereby eliminating the reliance on the honesty of external relays. The main **contributions** are as follows:

- *Asynchronous cross-chain mode:* We present the first asynchronous cross-chain mode suitable for proof-of-stake (PoS) public blockchains and permissioned blockchains, which can construct an honest majority committee. The security of cross-chain interactions is ensured by lightweight asynchronous cryptographic proofs committed by the committee. The committee cyclically provides AC-BaaS to enable efficient data interactions across multiple IoT domains in network-unstable environments.
- *Innovative cryptographic proofs:* We introduce a novel cryptographic primitive called delayed aggregate signature (DAS) to generate asynchronous cross-chain proofs (ACPs) underpinning IoT multi-domain data exchanges. DAS combines the ideas of aggregate signatures and verifiable delay functions (VDFs) to generate an aggregate signature for multiple transactions after a controlled delay and supports public fast verification. Compared to the traditional synchronous cross-chain mode, the cross-chain proof constructed using DAS significantly reduces both the proof size and the computational overhead. As the foundation of AC-BaaS, DAS is crucial for secure asynchronous cross-chain interactions, enabling the continuous generation of ACPs in asynchronous mode. We also conduct a detailed exploration of the ideal DAS delay size and how it should be flexibly set to trade-off security and efficiency in real-world application scenarios.
- *Restricted-readable consistency mechanism:* We design a transaction sequence guarantee mechanism to ensure the consistency of asynchronous cross-chain transactions. It maintains a multilevel buffer pool to coordinate the mainchain and sidechain for transaction ordering and confirmation, thus avoiding conflicts. The mechanism ensures the internal confidentiality of different IoT data domains, making the transaction sequence readable only to the authenticated leader. To optimize the utility of the buffer pool mechanism, we propose a heuristic algorithm for dynamically tuning the parameters to balance performance and resource consumption.

TABLE I
FEATURE COMPARISON WITH EXISTING SIDECHAINS

Schemes \ Features	Sidechain mode	Cross-chain proof[1]	Asynchronous network/mode	Batch commitment[2]	IoT data exchange[3]	Asynchronous CTx pool optimization	Trade-offs in delay settings
Pegged sidechains [9]	Parent-child	SPV proof	✗	✗	✗	-	-
PoW sidechains [10]	Parallel	NIPoPoWs	✗	✗	✗	-	-
PoS sidechains [11]	Parallel	ATMS	✗	✗	✗	-	-
Ref. [12], Ge-Co [15]	Parent-child	TSS	✗	✗	✗	-	-
AsyncSC [1]	Parallel	ACP	✓	✓	✓	✗	✗
AC-BaaS (This work)	Parallel	ACP	✓	✓	✓	✓	✓

[1] SPV stands for simplified payment verification; NIPoPoWs stands for non-interactive proofs of Proof-of-Work; ATMS stands for ad-hoc threshold multi-signatures; TSS stands for threshold signature schemes. [2] Each cross-chain proof in existing sidechains supports committing only a single transaction at a time. [3] Existing sidechains only support cross-chain exchanges involving cryptocurrencies.
[*] Explanation of symbols: ✓: Satisfied; ✗: Unsatisfied; -: Inapplicable.

• *Comprehensive evaluation:* We implement a prototype based on Hyperledger Fabric [18], and ChainMaker [19]and the evaluation results demonstrate that AC-BaaS outperforms SOTA solutions. It improves transaction throughput by an average of $1.71 \sim 5.09 \times$, reduces latency by 64.36% to 85.49%, enhances success ratio by a relative 50.26% to 142.74%, and maintains comparable overhead.

II. RELATED WORK

We classify the related work into three primary directions and provide a comparative analysis. (i) Since this paper proposes the first asynchronous sidechain-based scheme within permissioned blockchains, we first present the sidechain work in permissionless blockchains. (ii) Next, we compare the relay-based cross-chain interaction work commonly employed in permissioned blockchains. (iii) Finally, we analyze recent studies that utilizes cryptographic techniques to construct novel cross-chain proof primitives.

Permissionless sidechains: Back et al. [9] first proposed sidechain technology, which enables asset transfers to the Bitcoin blockchain via a pegged sidechain and maintains the security of the mainchain through the use of SPV proofs. RSK [20] introduces a Bitcoin sidechain offering Ethereum-compatible smart contract functionality. It designs a Powpeg bridge protocol to ensure secure asset transfers between Bitcoin and RSK.

Trusted relays: Most existing permissioned blockchains utilize trusted relays or relay chains for cross-chain interactions [13], [14], [18], [21], [22]. Cosmos [21] introduces a relay chain-based Inter-Blockchain Communication (IBC) protocol, which facilitates cross-chain interactions between heterogeneous blockchains and enhances interoperability through its Hub-and-Zone structure. RAC-Chain [22] proposes a permissioned cross-chain model based on asynchronous consensus and a relay chain architecture.

Cross-chain proof primitives: Although most of such approaches in this part also belong to the sidechain work within permissionless blockchains, their proposals are accompanied by the construction of a novel cross-chain proof primitive to replace the original SPV proofs. Kiayias et al. [10] proposed a Proof-of-Work (PoW) sidechain that employs non-interactive Proof-of-Work (NIPoPoW) proofs to achieve logarithmic proof size. Proof-of-Stake (PoS) sidechain [11] constructs ad-hoc threshold multi-signatures (ATMS) to achieve a lightweight proof size. Yin et al. [12], [15] employ threshold signatures to further reduce the proof size.

Comparison: We compare the features of AC-BaaS with existing SOTA sidechains in Table I.

For permissionless blockchains, sidechains mainly address the scalability of cryptocurrency cross-chain transfers. However, even as transaction processing efficiency improves, congested permissionless blockchains remain unsuitable for resource-constrained devices handling large-scale data transfer transactions. We propose AC-BaaS, which generates an asynchronous cross-chain proof (ACP) for multiple transactions for an epoch in batches while utilizing a committee to reduce the number of signatures. Thus, it supports cross-domain exchanges, including IoT data, in an asynchronous environment.

For permissioned blockchains, the introduction of trusted relays as a centralized cross-chain component carries the risk of a single point of failure. The relay must handle a large number of cross-chain interaction messages, adding complexity and dependency to the system, potentially leading to performance bottlenecks. This paper leverages a committee-based approach to provide AC-BaaS, which conforms to the permissioned blockchain trust model while avoiding privacy issues caused by external third-party agencies.

Existing cross-chain proof primitives are either improved based on SPV proofs, where the proof size remains large, or based on committee voting, which commits only to the correctness of cross-chain transactions, often overlooking stability and presenting security concerns. AC-BaaS designs DAS and proposes ACP based on DAS. It supports batch commitments to the correctness and stability of multiple cross-chain transactions. Thus, AC-BaaS significantly reduces proof size and computation overhead, improving the efficiency of proof generation and verification.

Our previous work, AsyncSC [1], proposed an alternative asynchronous sidechain architecture. Building upon AsyncSC, AC-BaaS further optimizes the design of the asynchronous cross-chain transaction pool and introduces a heuristic algorithm for dynamic parameter tuning to balance system performance and resource consumption. Moreover, AC-BaaS provides a detailed analysis of the optimal DAS delay and investigates how to flexibly configure the DAS delay in practical deployment scenarios to achieve a trade-off between security requirements and system efficiency.

III. AC-BAAS: SYSTEM OVERVIEW

This section introduces the system model and assumptions of AC-BaaS. We recommend that readers refer to Appendix A,

TABLE II
SUMMARY OF MAIN NOTATIONS

Notation	Description
MC	Mainchain or source blockchain
SC	Sidechain or target blockchain
CTx_i	The i-th cross-chain transaction
$\mathsf{CTx}_i^{\mathsf{send}}$	The send intra-chain transaction of CTx_i on MC
$\mathsf{CTx}_i^{\mathsf{recv}}$	The receive intra-chain transaction of CTx_i on SC
$\mathsf{L}_{\mathsf{MC}}/\mathsf{L}_{\mathsf{SC}}$	Ledger of MC/SC after block stabilization
CTxSet	A packed set of CTxs within an epoch, *i.e.*, $\{\mathsf{CTx}_i\}\|_{i=1}^n$
C_i	The i-th member of an IoT domain committee
sk/pk	Private/public key for transaction signing/verifying
σ	A signature of a transaction
σ_{DAS}	The delayed aggregate signature for a CTxSet
ek/vk	Public parameters of DAS, *i.e.*, evaluation/verification key

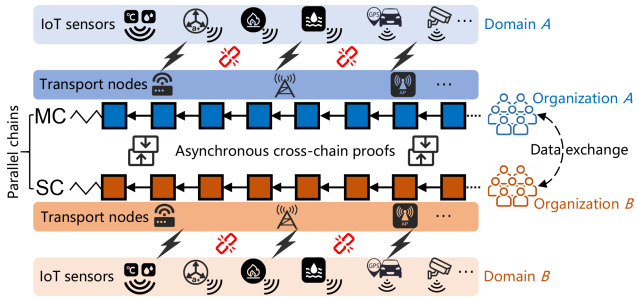


Fig. 2. System model of AC-BaaS.

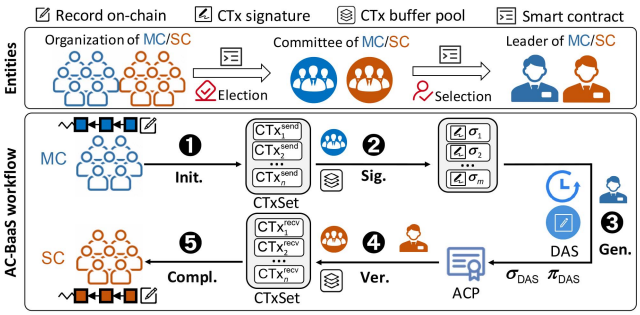


Fig. 3. Cross-chain interaction between two AsyncSCs.

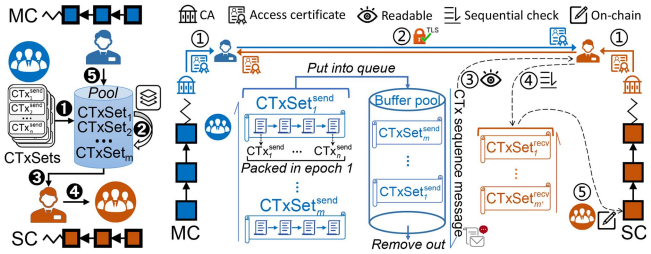
available in the online for background and preliminaries on AC-BaaS. Key notations are shown in Table II.

A. System Model

This paper focuses on multi-domain data exchange scenarios in IoT, as illustrated in Fig. 2. IoT sensor data is uploaded to permissioned blockchains managed by different organizations via transport nodes like IoT gateways and base stations for decentralized management. Independent blockchains in a parallel chain relationship require cross-chain interaction for data exchange among different organizations. Due to resource constraints of IoT devices and unstable network connections, cross-chain communication is achieved by generating and verifying asynchronous cross-chain proofs (ACPs).

Next, we describe the entities involved in cross-chain interactions and the workflow depicted in Fig. 3.

Entities: Each AsyncSC comprises three types of entities:



(a) Checking steps. (b) CTx sequence restricted readable.

Fig. 4. Asynchronous transaction sequence guarantee mechanism.

Organization: It consists of permissioned blockchain nodes S^{Org} that maintain data from different IoT domains. These nodes may be elected to a committee to provide AC-BaaS, which records data from IoT devices on-chain and facilitates cross-chain interactions.

Committee: It is a set of nodes $S^C = \{C_i\}|_{i=1}^m \in S^{Org}$ that are fairly elected from the organization by periodically running an election protocol[1] via smart contracts. These nodes provide AC-BaaS, verify and sign transactions $\{\mathsf{CTx}_i\}|_{i=1}^n$ packed per epoch, and generate ACPs by the leader. Additionally, the committee maintains a buffer pool to ensure the consistency of asynchronous CTxs.

Leader: It is a node C^L with the highest election value in the committee, i.e., $C^L \in S^C$ and $C^L \leftarrow \mathtt{max.ElecVal}(\{C_i\}|_{i=1}^m)$. It is responsible for performing delayed signature aggregation of the CTx set $\mathsf{CTxSet} := \{\mathsf{CTx}_i\}|_{i=1}^n$ for each epoch of the committee and generating ACPs. Additionally, it verifies the received ACPs and finalizes the corresponding CTxs.

Workflow: To accommodate the asynchronous characteristics of IoT networks, the asynchronous cross-chain interaction process in AC-BaaS is decoupled into five steps, where the execution of CTxs partially overlaps in time, thereby enabling concurrency in intermittent IoT environments (see Appendix A-B, available in the online for details):

❶ CTx *initiation:* Suppose in an epoch, the organization to which MC belongs initiates a batch of CTxs, i.e., $\{\mathsf{CTx}_i\}|_{i=1}^n$. The nodes of MC start to record $\{\mathsf{CTx}_i^{\mathsf{send}}\}|_{i=1}^n$ on-chain. At the same time, the committee of MC receives a CTxSet message formed by packing these CTxs, whereupon committee members asynchronously monitor the status responses of the CTxs in batch mode, guided by the embedded CTxSet_j identifier.

❷ *Committee signing:* The committee of MC starts to provide AC-BaaS. First, the committee puts the transactions of CTxSet into the buffer pool. Then, each committee member $C_i \in S^C$ signs the CTxSet using his/her private key via any EUF-CMA-secure digital signature scheme, i.e., $\sigma_i \leftarrow \mathtt{Sig}(\mathsf{CTxSet}, sk_i)$ for $i = 1, \ldots, m$. Finally, the committee sends a sequence of $\{(pk_i, \sigma_i)\}|_{i=1}^m$ to the leader of MC.

❸ *ACP generation with delay:* The leader of MC executes a delayed signature aggregation on the committee's signatures, i.e., $(\sigma_{\mathsf{DAS}}, \pi_{\mathsf{DAS}}, \cdot) \leftarrow \mathtt{DASig}(\cdot)$. Here, the $\cdot$ notation indicates that some inputs or outputs of the algorithm are omitted (details in Section IV-A). The outputs of DASig along with the public parameter pp form an ACP representing the validity commitment

[1]Existing committee election schemes are well established, as noted in the Ref. [15], [23], [24], and their optimization is part of our future work.

of CTxSet on MC. DASig introduces a controllable delay δ by performing a sequence of consecutive computations, thereby restricting the finality recovery of $\{\mathsf{CTx}_i^{\mathsf{recv}}\}|_{i=1}^{n}$ by MC and adversaries. The delayed ACP output serves as a commitment to the stability of the CTxSet, enabling the MC to initiate pipelined parallel processing for the next epoch of CTxs.

❹ *ACP verification:* Upon receiving the ACP from MC, the leader of SC verifies the validity of the ACP, i.e., $1/0 \leftarrow \mathtt{DASVer}(\cdot)$. Upon successful verification, the committee of SC queues the transactions within CTxSet into the buffer pool awaiting on-chain recording. To ensure the consistency of asynchronous CTxs, the SC committee conducts a transaction sequentiality check utilizing the buffer pool maintained by MC to prevent conflicts. In cases of anomalous delay or loss in in MC's ACP transmission, such as when SC's response to CTxSet_j exceeds Δ_{async}, the SC buffer pool suspends dependent CTxs. Upon receipt of the retransmitted ACP from MC, the SC leader merges the transaction pools and reorders them via sequence numbers.

❺ CTx *completion:* The organization to which SC belongs records the $\{\mathsf{CTx}_i^{\mathsf{recv}}\}|_{i=1}^{n}$ in CTxSet on-chain. Once these transactions are stabilized within $\mathsf{L_{SC}}$, the corresponding CTxs for this epoch are finalized.

B. Assumptions

Assumptions: The upper bound Δ on message transmission delay due to asynchronous communication may be unknown. In practice, we assume a negligible probability of $\Delta = \infty$ between MC and SC. To ensure safety and liveness, MC retransmits the CTxs to SC if they exceed an asynchronous response time threshold Δ_{async}. The CTxs of each epoch eventually complete their execution through the AC-BaaS provided by the committee. If the verification of MC's cross-chain proof passes, then SC will accurately complete the corresponding cross-domain data on-chain and stabilize it. To minimize the overhead caused by the committee elections, we assume a long-term election period, with each period containing multiple epochs for packing transactions.

Threat model: We consider that an adversary may attempt to violate the protocol by forging signatures or delaying/denying message transmission. However, the adversary's behavior is rational, i.e., it weighs the costs and benefits of the attack. As in existing blockchain systems, we assume that the nodes maintaining the permissioned blockchains are majority honest [12]. The adversary does not possess enough computational power to break the participants' signatures or compromise blockchains and smart contracts.

IV. Delayed Aggregate Signature

In this section, we introduce a novel cryptographic primitive known as the delayed aggregate signature (DAS), which serves as the fundamental building block of ACP and the core of AC-BaaS. DAS integrates properties from aggregate signatures and VDFs. It provides a way to: (i) Aggregate multiple individual signatures $\{\sigma_i\}|_{i=1}^{m}$ into a single aggregate signature σ_{DAS} after a controllable delay δ and produce a proof of delay π_{DAS}. (ii) Enable the verifier to quickly and publicly verify the σ_{DAS} and π_{DAS}, thereby effectively verifying the authenticity of $\{\sigma_i\}|_{i=1}^{m}$ and the delay δ.

A. Definition of Delayed Aggregate Signature

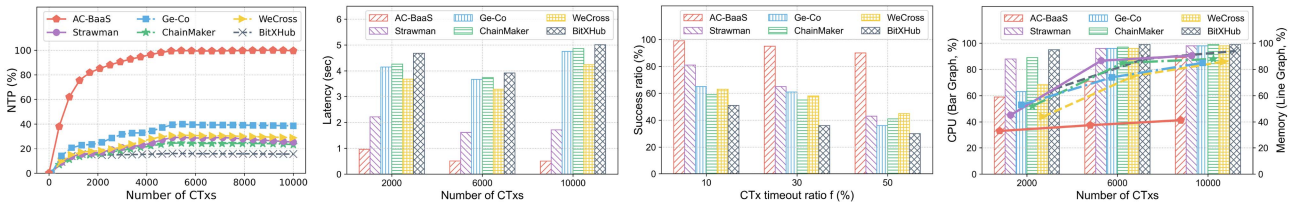
Definition 1 (Delayed Aggregate Signature): A delayed aggregate signature is a tuple of algorithms $\Pi = (\mathtt{ParGen}, \mathtt{KeyGen}, \mathtt{Sig}, \mathtt{Ver}, \mathtt{KeyAgg}, \mathtt{AKCheck}, \mathtt{DASig}, \mathtt{DASVer})$, whose syntax is described below:

- $(sp, pp) \leftarrow \mathtt{ParGen}(1^{\lambda}, t)$: A parameter generation algorithm that takes as input the security parameter λ and the delay parameter t, and outputs the global system parameters sp and the public parameters $pp = (ek, vk)$ consisting of an evaluation key ek and a verification key vk.
- $(pk_i, sk_i) \leftarrow \mathtt{KeyGen}(sp)$: A key generation algorithm that inputs sp and generates a public-private key pair (pk_i, sk_i) for the invoker.
- $\sigma_i \leftarrow \mathtt{Sig}(sk_i, M)$: A common signature algorithm that inputs the private key sk_i and a message $M \in \{0,1\}^*$, and outputs a signature σ_i.
- $1/0 \leftarrow \mathtt{Ver}(M, pk_i, \sigma_i)$: A signature verification algorithm that inputs the message M, the public key pk_i and the signature σ_i, and outputs 1 (accept) if the verification is successful and 0 (reject) otherwise.
- $apk \leftarrow \mathtt{KeyAgg}(PK)$: A key aggregation deterministic algorithm that inputs a set of public keys $PK = \{pk_i\}|_{i=1}^{m}$ and outputs an aggregate public key apk.
- $1/0 \leftarrow \mathtt{AKCheck}(PK, apk)$: An aggregate key checking algorithm that inputs a public key set PK and an aggregate public key apk, and outputs 1 if the recomputation check is correct and 0 otherwise.
- $(\sigma_{\mathsf{DAS}}, \pi_{\mathsf{DAS}}, M') \leftarrow \mathtt{DASig}(M, PK, \{(pk_i, \sigma_i)\}|_{i=1}^{m}, ek)$: A delayed aggregation signature algorithm that inputs the message M, the set of public keys PK, a sequence of public key-signature pairs $\{(pk_i, \sigma_i)\}|_{i=1}^{m}$ and the evaluation key ek, and outputs a delayed aggregate signature σ_{DAS}, a proof of delay π_{DAS} and a unique output M' corresponding to message M.
- $1/0 \leftarrow \mathtt{DASVer}(M, apk, \sigma_{\mathsf{DAS}}, \pi_{\mathsf{DAS}}, M', vk)$: A delayed aggregate signature verification algorithm that inputs the message M, the aggregate public key apk, the delayed aggregate signature σ_{DAS}, the delayed proof π_{DAS}, the unique output M' of the message M and the verification key vk, and outputs 1 if the verification is successful and 0 otherwise.

B. Realization of Delayed Aggregate Signature

The specific realization of DAS is partially inspired by two primitives: aggregate signatures and VDFs, both initially proposed by Boneh et al. [25], [26]. Aggregate signatures or multi-signatures have been improved by Ref. [27], [28], while VDFs have been enhanced by Ref. [29]. It is noteworthy that the improvements on these primitives are independent of this paper. We extract the signature compression property of aggregate signatures and the delay verifiability property of VDFs to implement compact ACPs.

We refer to the detailed constructions in the multi-signature scheme MGS [30] and the VDF scheme for incremental verifiable computation (IVC) [26] to realize the DAS scheme. We recall some of the necessary preliminaries. Let $P = \{P_1, \ldots, P_m\}$ be a group of players. Let $\mathbb{G}$ be a Gap Diffie-Hellman (GDH) group of prime order p, and g be a generator of $\mathbb{G}$. Let H be a collision-resistant hash function from $\{0,1\}^*$ to $\mathbb{Z}_p^*$. $\mathcal{V}_{DDH}(\cdot)$ is an efficient algorithm that can solve the Decisional Diffie-Hellman (DDH) problem in $\mathbb{G}$. For IVC, we can construct an



(a) Throughput vs. the # of CTxs. (b) Latency vs. the # of CTxs. (c) Success ratio vs. the timeout ratio. (d) Overhead vs. the # of CTxs.

Fig. 5. Performance comparison of different baselines across various metrics.

IVC proof system via any succinct non-interactive argument of knowledge (SNARK) system (e.g., Ref. [31]). The function $f_\lambda : \mathbb{N} \times X_\lambda \to X_\lambda$ is defined by $f_\lambda(k, x) = g_\lambda^{(k)}(x)$, where g_λ is a (s, ϵ)-sequential function on an effectively sampleable domain of size $O(2^\lambda)$. For conciseness, we omit the iterative computation of SNARK from the description.

We present the construction of DAS below.

- $\underline{\mathtt{ParGen}(1^\lambda, t)}$: $sp := (p, g, H)$. Compute $(ek, vk) \xleftarrow{R} \mathtt{IVCGen}(\lambda, f_\lambda)$. Let k be the largest integer that makes the time required for $\mathtt{IVCProve}(ek, k, M)$ less than t. Return $(sp, pp) := ((ek, k), (vk))$.
- $\underline{\mathtt{KeyGen}(sp)}$: For each player P_i, compute $x_i \xleftarrow{R} \mathbb{Z}_p^*$, $y_i \leftarrow g^{x_i}$, and return $(pk_i := (p, g, H, y_i), sk_i := x_i)$.
- $\underline{\mathtt{Sig}(sk_i, M)}$: Compute $\sigma_i \leftarrow H(M)^{sk_i}$, and return σ_i.
- $\underline{\mathtt{Ver}(M, pk_i, \sigma_i)}$: Parse pk_i as (p, g, H, y_i). Return 1 if $\mathcal{V}_{DDH}(g, y_i, H(M), \sigma_i) = 1$, and return 0 otherwise.
- $\underline{\mathtt{KeyAgg}(PK)}$: Return $apk \leftarrow \Pi_{i=1}^m(pk_i)$.
- $\underline{\mathtt{AKCheck}(PK, apk)}$: Return 1 if $apk = \Pi_{i=1}^m(pk_i)$, and return 0 otherwise.
- $\underline{\mathtt{DASig}(M, PK, \{(pk_i, \sigma_i)\}|_{i=1}^m, ek)}$: Compute $\sigma_{\mathsf{DAS}} \leftarrow \Pi_{i=1}^m(\sigma_i)$, and $(\pi_{\mathsf{DAS}}, M') \xleftarrow{R} \mathtt{IVCProve}(ek, k, M)$. Return $(\sigma_{\mathsf{DAS}}, \pi_{\mathsf{DAS}}, M')$.
- $\underline{\mathtt{DASVer}(M, apk, \sigma_{\mathsf{DAS}}, \pi_{\mathsf{DAS}}, M', vk)}$:

  Return 1 if $\mathcal{V}_{DDH}(g, apk, H(M), \sigma_{\mathsf{DAS}}) = 1$

  and $\mathtt{IVCVerify}(vk, M, M', k, \pi_{\mathsf{DAS}}) = 1$,

  and return 0 otherwise.

To maintain the logical structure of the paper, we present the security proof of the DAS construction in Section VI. The application of DAS to AC-BaaS is detailed in Appendix B, available in the online.

C. DAS Compatibility Discussion

As a cryptographic primitive, DAS is well-suited for constructing asynchronous cross-chain proofs. Although DAS proofs differ structurally from traditional proofs such as SPV, existing SPV-based synchronous cross-chain systems can be adapted to accommodate DAS with minimal modifications, namely through node upgrades to support the underlying cryptographic functions, thereby harnessing the efficiency gains from asynchronous batch interactions. Concurrently, these systems can retain SPV proof capabilities to ensure backward compatibility or synchronous fallback.

Integrating SPV can further augment the verification of DAS outcomes. Upon receipt of an ACP generated by DAS, the target blockchain may optionally invoke SPV to confirm that sampled transactions from the committed epoch are indeed finalized within the source blockchain's headers. This hybrid approach empowers organizational auditors to perform random spot-checks on transaction samples, thereby enhancing oversight of cross-chain robustness and integrity.

In practice, cross-chain verification logic can dynamically switch based on prevailing network conditions. Pure synchronous SPV configurations may impose inherent limitations, wherein the DAS-induced delay (δ) could inadvertently prolong processing times. To mitigate this, hybrid implementations can adaptively adjust per scenario, trading off security margins for improved efficiency (see Section V-B).

V. System Details of AC-BaaS

In this section, we describe the asynchronous transaction sequence guarantee mechanism and discuss the details of the delay settings of the DAS.

A. Transaction Sequence Guarantee Mechanism

1) Buffer Pool-Based Implementation: To maintain the consistency of asynchronous CTxs, as shown in Fig. 4, we propose a restricted-readable transaction sequence guarantee mechanism. This mechanism can be implemented in various ways; here, we provide a buffer pool-based approach.

The sequential check consists of five main steps, as shown in Fig. 4(a). ❶ *Receiving*. When the MC's committee receives the CTxSet for each epoch, its leader maintains these CTxs in the local multilevel buffer pool. To improve efficiency and minimize conflicts, we draw inspiration from Ref. [32]. The multilevel buffer pool uses an adaptive heap structure to categorize and store transactions according to different priorities (e.g., timestamp, importance, dependency). The adaptive heap dynamically adjusts the order based on the current load situation and the dependencies of the CTxs, ensuring that CTxs with high priority and early timestamps are located at the top of the heap. ❷ *Ordering*. The MC leader dynamically adjusts the order of the pooled CTxs to optimize processing efficiency, using a sliding window approach in micro-batch processing. It also dynamically adjusts the window size and movement speed based on real-time transaction traffic. ❸ *Processing*. The leader of SC performs conflict checking on the CTxs in each CTxSet received asynchronously based on the buffer pool maintained by MC, and then sends the processed set of CTxs to the committee for confirmation. ❹ *Confirming*. The committee of SC uses smart

contracts to automate the confirmation of the transaction order and ensure consistency through BFT consensus algorithms. After confirming that there are no conflicts, the relevant CTxs are executed in an orderly manner. ❺ *Removing*. Finally, after confirming that the CTxs have stabilized through a persistent query on the ledger, the leader of MC removes the executed CTxs from the buffer pool.

2) Restricted Readable Mechanism: In the context of this paper, to adhere to the internal confidentiality of permissioned blockchains, the CTx sequence maintained by the buffer pool is restricted to be readable only by authenticated leaders, as shown in Fig. 4(b). ① Upon leader nodes' access to the permissioned blockchains, certificate authorities (CAs) issue access certificates to them. ② When cross-chain interactions are required, leaders on both sides exchange certificates over a secure communication channel such as TLS to authorize each other. ③ Then, the leader of SC can receive a signed message from MC containing only the transaction sequence in the buffer pool. ④ The leader of SC then performs a sequential check on the received CTxs based on this message. ⑤ The committee of SC completes the on-chain recording of CTxs after checking.

3) Utility Optimization: Currently, research on transaction pools (or mempools) for blockchains primarily focuses on optimizing a single blockchain [33]. However, cross-chain transaction pool mechanisms are inherently more complex than those within a single blockchain. Moreover, the research and optimization of cross-chain transaction pools in asynchronous environments remains a significant gap.

To optimize the buffer pool-based sequence guarantee mechanism for CTxs, the problem is transformed into a multi-objective optimization problem. The optimization objectives include maximizing system throughput, minimizing latency and conflict rate, and maintaining balanced system resource utilization. To this end, we propose a heuristic approach to solve the problem.

Maximizing throughput: Here, throughput (Thr) represents the cumulative count of successfully processed transactions across all micro-batches within a designated time window. Maximizing Thr is crucial for enhancing the system's overall performance. The optimization objective can be expressed as:

$$\text{Maximize } \texttt{Thr} = \frac{1}{t_W} \sum_{k=1}^{M_W} \frac{N_k}{\sum_{i=1}^{N_k} (Q(T_i) + P(T_i))},$$

where t_W denotes the overall time window length. M_W represents the number of micro-batches within this window. In the k-th micro-batch, N_k indicates the number of processed transactions. The queuing delay and processing time of transaction T_i are denoted by $Q(T_i)$ and $P(T_i)$, respectively.

Minimizing latency: The delay here for each transaction T_i is composed of the queuing delay $Q(T_i)$ and the processing time $P(T_i)$. Minimizing the average transaction delay (Del) for all transactions is essential to enhancing system responsiveness:

$$\text{Minimize } \texttt{Del} = \frac{1}{\sum_{k=1}^{M_W} N_k} \sum_{k=1}^{M_W} \sum_{i=1}^{N_k} (Q(T_i) + P(T_i)).$$

Minimizing conflict ratio: Reducing transaction conflict probability improves the overall transaction success ratio. Accordingly, the optimization objective is to minimize the average conflict ratio (Conf) across all transactions:

$$\text{Minimize } \texttt{Conf} = \frac{1}{\sum_{k=1}^{M_W} N_k} \sum_{k=1}^{M_W} \sum_{i=1}^{N_k} C(T_i),$$

where $C(T_i)$ denotes the conflict probability of transaction T_i.

Balancing utility: We integrate the factors influencing system performance (throughput, latency, and conflict ratio) and resource consumption (buffer pool size, sliding window size, and movement speed) to formulate a cost/utility function (Cost). This function represents the utility balance between system performance and resource consumption. The objective is to minimize this cost function:

$$\begin{aligned}\text{Minimize } \texttt{Cost}(B, W, v) = {} & \lambda_1 \cdot (-\texttt{Thr}) + \lambda_2 \cdot \texttt{Del} \\ & + \lambda_3 \cdot \texttt{Conf} + \lambda_4 \cdot (\alpha \cdot B + \beta \cdot W + \gamma \cdot v),\end{aligned}$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are the weighting coefficients that control the importance of each objective. B denotes the size of the buffer pool queue, W represents the size of the sliding window, and v is the movement speed of the sliding window. α, β, and γ are the weights representing the impact of buffer pool size, sliding window size, and window movement speed on resource consumption.

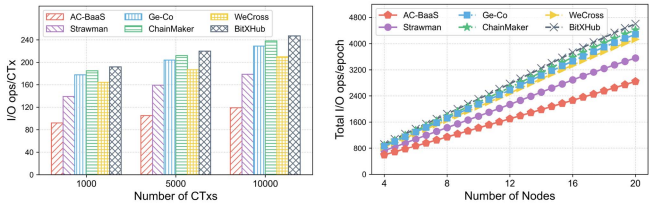
We provide a further description of the cost function. First, higher system throughput indicates better performance, so in the optimization process, we aim to maximize throughput. To align this with the minimization objective, we take the negative of the throughput in the cost function. Second, lower latency and conflict ratios contribute to better system performance, so they are minimized. Regarding resource consumption, which is primarily determined by buffer pool size, sliding window size, and movement speed, these are minimized to reduce resource wastage. The final cost function integrates both system performance and resource consumption, with the optimization objective of balancing the two by applying weighting coefficients $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ to find the optimal solution.

Constraints: In this multi-objective optimization problem, the mechanism must satisfy a set of constraints to guarantee system operability and resource utility. First, the buffer pool queue size B, the sliding window size W and the window movement speed v must satisfy $B_{\min} \leq B \leq B_{\max}$, $W_{\min} \leq W \leq W_{\max}$, and $v_{\min} \leq v \leq v_{\max}$, respectively, to guarantee appropriate resource allocation and prevent both overloading and underutilization. In addition, to ensure the system's response speed and subject to the Δ_{async} constraint, the queuing delay $Q(T_i)$ must satisfy $Q(T_i) \leq Q_{\max}$. The transaction conflict ratio $C(T_i)$ must satisfy $C(T_i) \leq C_{\max}$ to preserve system efficiency. Finally, the total resource consumption constraint is $g(B, W, v) \leq \texttt{Res}_{\max}$ to ensure the system does not exceed its available resources, thereby avoiding performance degradation. Here, $g(B, W, v)$ is a resource consumption function that incorporates B, W, and v. The $\texttt{Res}_{\max}$ represents the upper bound of resources available to each buffer pool.

Utility optimization problem solving: The difficulty in solving the above problem involves multi-objective optimization, which includes trade-offs among throughput, delay, conflict ratio, and resource consumption, and also entails combinatorial optimization of discrete and continuous variables, such as queue size and window size. Furthermore, due to the uncertainty caused by the dynamic adjustment of resources, the problem is not able to find the global optimal solution in all cases by polynomial

TABLE III
THROUGHPUT OF AC-BAAS AND BASELINES

Schemes	Strawman	Ge-Co	ChainMaker	WeCross	BitXHub	**AC-BaaS**
Avg. TPS	457.93	652.70	402.92	497.40	290.82	**1770.27**
Avg. increase	2.87 ×	1.71 ×	3.39 ×	2.56 ×	5.09 ×	-



(a) I/O overhead vs. the # of CTxs. (b) CTx arrival rate = 2000 CTx/sec.

Fig. 6. System I/O overhead evaluation.

time algorithms, which is of NP-hard difficulty. To this end, we propose a heuristic approach that offers essential insights for solving the problem. This method emphasizes buffer pool state feedback and dynamically tunes transaction allocation policies to achieve rapid convergence and optimal solutions for multi-objective demands.

Algorithm 1 (see Appendix C, available in the online) provides a detailed pseudocode description for the heuristic utility optimization solution. Its goal is to optimize the effectiveness of the buffer pool mechanism through iterative parameter adjustments until convergence. Before delving into the line-by-line details of the algorithm, we first outline the fundamental principles of the heuristic dynamic adjustment. This strategy harnesses real-time system load feedback to fine-tune parameters, including the buffer pool queue size B and the sliding window length W. For instance, when the number of queued CTxs surpasses the threshold $\mathtt{Td}_B$ (e.g., 80% of total capacity), B and W are proportionally expanded to accommodate surges, while prioritizing associated transactions via adaptive heap weights. Conversely, during low-load periods, these sizes are scaled down to conserve resources. This methodology builds upon our established adaptive heap and utility optimization frameworks, thereby guaranteeing equilibrated performance.

Input parameters are initialized based on a generally balanced system configuration, including multi-objective importance weights $\lambda_1, \lambda_2, \lambda_3, \lambda_4$, resource impact weights α, β, γ, and the time window length t_W. The algorithm establishes the initial buffer pool parameters B, W, v (line 4) before commencing iterative optimization. In each time window t_W, the algorithm collects feedback indicators via FB(), such as the number of transactions in the queue (numCTx), delay (Del), conflict ratio (Conf), and resource usage (resUsage) (line 10). Subsequently, the algorithm adaptively adjusts weights and parameters (lines 11-33). If numCTx reaches the buffer pool's threshold $\mathtt{Td}_B$, the algorithm proportionally increases the throughput and resource consumption weights in the cost function. Given the negative contribution of throughput in the cost function, λ_1 is reduced accordingly (line 13). Additionally, the algorithm increases the buffer pool queue size and window length (line 14). Conversely, the algorithm performs inverse adjustments (lines 15-17). When latency, conflict rate, or resource usage exceeds predefined thresholds, the algorithm similarly adjusts the relevant parameters (lines 19-33). By this stage, the algorithm determines an optimized set of weights $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ and parameters B, W, v.

In each iteration round, the algorithm performs a heuristic search to explore the optimal parameter combination based on the current weights and parameters determined in previous steps. During micro-batch processing, the algorithm initially sorts transactions using adaptive heap sorting (lines 36-46) and adjusts the resource weights (lines 47-62) based on current performance metrics. $\mathtt{prio}(T_i)$ denotes the priority of transaction T_i, while p_1, p_2, p_3 represent the priority coefficients for transaction importance, timestamp, and dependency, respectively (line 39). The algorithm then performs a neighborhood update (lines 63-89), which generates a set of neighborhood solutions (NBHD) through minor adjustments ($\pm\epsilon$) to the buffer pool parameters B, W, v of the current solution. It computes the cost of each neighborhood solution and selects the optimal solution as the current one (line 70). Additionally, the algorithm employs a greedy strategy. If the cost of the current neighborhood solution is lower, the parameters of the current solution are updated (lines 72-78). Next, the algorithm performs a convergence check (lines 79-88) by calculating the change in the cost of the current solution relative to the previous one and verifying whether the change is below a predefined threshold. If the change is less than ϵ over successive $\mathtt{Td}_{\mathtt{Conv}}$ iterations, the algorithm is deemed to have converged, and the utility optimization process is terminated. After updating the neighborhood parameters, the algorithm assigns resources to transactions and processes them in priority order (lines 90-93). Finally, it outputs the optimized cost, resource weights, and utility parameters.

Complexity: Overall, the runtime complexity of our heuristic solution algorithm is $\mathcal{O}(\sum_{k=1}^{M_W} N_k \cdot \log N_k + \sum_{k=1}^{M_W} N_k)$. It is primarily divided into the $\mathcal{O}(\sum_{k=1}^{M_W} N_k \cdot \log N_k)$ adaptive heap operation and the $\mathcal{O}(\sum_{k=1}^{M_W} N_k)$ heuristic search steps. Here, M_W denotes the number of iterations in the sliding window, and N_k represents the number of transactions per micro-batch. The algorithm's complexity is approximately linear with respect to the total number of transactions $\sum_{k=1}^{M_W} N_k$. Specifically, in scenarios with smaller micro-batches ($N_k \ll \sum_{k=1}^{M_W} N_k$), the algorithm's complexity approaches linearity. Conversely, in cases with larger micro-batches ($N_k \approx \sum_{k=1}^{M_W} N_k$), where nearly the entire set of transactions is processed within a single sliding window, the algorithm exhibits sub-linear complexity.

B. Delay Settings for DAS

1) Analysis of DAS Delay: The core idea of implementing DAS lies in the fact that the DASig algorithm produces a controlled delay by performing a series of sequential computations on the input for a given number of steps, such as the iterative computation of the SNARK utilized in the IVC adopted in this paper. At the same time, it ensures that the results can be verified quickly. Under given hardware and algorithmic conditions, the time required to compute DASig for a given length of input is fixed. This predictability of computation time allows the system designers to set specific delays as needed. We ensure that each computation goes through the same required steps and time by choosing the appropriate delay parameter t in the DAS's parameter generation phase.

With the workflow of Fig. 3 in mind, we now detail the settings for the size of the delay generated by the DAS. Our

TABLE IV
COMPARISON OF CROSS-CHAIN PROOFS

Schemes	PoW sidechains [10]	PoS sidechains [11]	Ge-Co [15]	**AC-BaaS**
Proof size[1]	$\mathcal{O}(\log cl)$	$\mathcal{O}(k)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Comp. cost[2]	$\mathcal{O}(\log cl)$	$\mathcal{O}(k)$	$\mathcal{O}(\lvert S^C\rvert)$	$\mathcal{O}(\frac{\lvert S^C\rvert}{n})$
Gen. time[3]	1205.12 s	54.54 s	3.91 s	**512.3 ms**
Ver. time	30.97 s	24.72 s	618.37 ms	**83.6 ms**

[1] The symbol cl denotes the chain length, and k is the common prefix parameter.
[2] The $|S^C|$ denotes committee size, and n denotes the number of CTxs in an epoch.
[3] Measure the time to generate cross-chain proofs for an epoch containing 1,200 CTxs. We set $cl \approx 2.03 \times 10^7$ (as of July 2024 on ETH); $k = 2160$; and a committee size of 10.

goal is to ensure that when SC receives the ACP, the delay δ generated by running DASig guarantees that the CTxSet, i.e., all of the CTxs in an epoch, have stabilized on MC. Thus, the key is to make the last transaction of this epoch on-chain stable. We now discuss the size of the ideal delay δ. These CTxs are recorded on MC in Fig. 3-❶. Assume that the last CTx on-chain in this epoch is of length $t_{LastCTx}$ from the last time slot of the epoch. Thus, $t_{LastCTx}$ is included in the duration required for the stabilization of CTxSet. Additionally, the DASig algorithm is started in Fig. 3-❸, i.e., the time for the committee to carry out the signatures in Fig. 3-❷, t_{Sig}, is already accounted for in the length of time required for the stabilization of CTxSet. Let the time required for a transaction to stabilize on the permissioned blockchain be t_{Stab}, then the ideal delay $\delta = t_{Stab} - t_{LastCTx} - t_{Sig}$. Since t_{Stab} and t_{Sig} are essentially constant in a specific permissioned blockchain and $t_{LastCTx}$ is easily obtained, it is feasible to set a suitable delay parameter t to control δ.

2) Trade-Offs in Delay Settings: In practice, the delay settings of DAS involve a trade-off between security and efficiency. Typically, δ is set slightly longer than its ideal duration to enhance security. Moreover, if the transaction volume on MC surges or intra-chain communication quality declines, block stabilization may be delayed, necessitating a longer δ. For different blockchains, δ is tailored to the specific durations of ledger state evolution [34].

Next, we delve deeper into the trade-offs involved in delay settings. To enhance flexibility in balancing security and efficiency across different scenarios, we introduce the concept of a security margin. In the computer field, a security margin is defined as the protective threshold allocated to exceed normal operational requirements in response to potential threats, failures, or malicious behaviors [35]. In this paper, the security margin is presented as a dynamically adjustable parameter within latency settings, designed to balance security and efficiency in asynchronous cross-chain interactions, particularly under varying network conditions or transaction loads. We define the security margin Δ_{SM} as an additional time adjustment to the asynchronous cross-chain delay δ that compensates for potential network jitter or transaction fluctuations, ensuring reliable cross-chain interactions through a moderate time buffer. The delay setting formula can be extended as:

$$\delta = t_{Stab} - t_{LastCTx} - t_{Sig} + \Delta_{SM}.$$

According to the security requirements of different application scenarios, we classify the security margin Δ_{SM} into three tiers: (i) *Ordinary.* This level applies to scenarios requiring high real-time responsiveness but lower security requirements, where response speed and system throughput take precedence. The value of Δ_{SM} is minimized to maximize operational efficiency. For instance, this includes environmental data sharing in smart cities. (ii) *Balancing.* This level suits typical application scenarios where a balance between security and efficiency is essential. The Δ_{SM} value is moderate to ensure adequate security without major compromises to operational efficiency. For instance, daily energy transactions in energy management exemplify such scenarios. (iii) *Sensitive.* This level addresses highly sensitive or privacy-intensive scenarios prioritizing data consistency and security. These scenarios require larger Δ_{SM} values to establish stronger security buffers. For example, cross-organizational data sharing in healthcare represents such scenarios.

The formula for dynamically adjusting the DAS delay (Δ_{SM}) is expressed as follows:

$$\Delta_{SM} = \omega_1 \cdot R_{net} + \omega_2 \cdot R_{tx} \cdot t_{Stab}.$$

The parameter R_{net} represents the degree of variation in network delay and jitter, which is determined using the following formula:

$$R_{net} = \frac{|\Delta_{delay}| + |\Delta_{jitter}|}{t_{obs}},$$

where Δ_{delay} and Δ_{jitter} represent the variation in average delay and delay jitter, respectively, measured within the observation window t_{obs}.

The parameter R_{tx} quantifies the magnitude of change in the system's transaction load over the observation window t_{obs} and is calculated as follows:

$$R_{tx} = \frac{|N_{t_2} - N_{t_1}|}{t_{obs}},$$

where N_{t_1} and N_{t_2} denote the transaction volumes at the start and end of the observation window t_{obs}, respectively.

The weight coefficients ω_1 and ω_2 determine the proportionate impact of changes in network communication and transaction volume on Δ_{SM}, respectively, and control the size of Δ_{SM} relative to δ.

VI. SECURITY ANALYSIS

We first provide a security proof for our DAS construction, and then define and prove the security of AC-BaaS. Due to space limitations, readers are referred to Appendix D, available in the online.

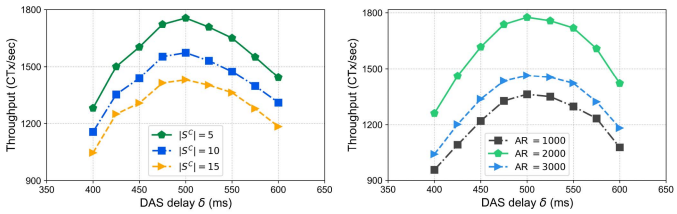
VII. EVALUATION

A. Implementation

To evaluate AC-BaaS, we implement a prototype based on two well-known permissioned blockchains: Hyperledger Fabric [18] and ChainMaker [19], using the Go language. We realize DAS by modifying the multi-signature MuSig2[2] and Harmony's VDF.[3] The prototype code of AC-BaaS will be released to the public in the near future.

Baselines: For a fair comparison, the baselines for evaluating AC-BaaS include two cross-chain prototypes we developed and three enterprise-level projects supporting cross-chain interactions on permissioned blockchains. Their names and core ideas are as follows: (i) *Strawman*. An incomplete AC-BaaS prototype using SPV proof for each CTx as a commitment. (ii) *Ge-Co* [15].

[2] https://github.com/aureleoules/musig2-coordinator
[3] https://github.com/harmony-one/vdf



(a) CTx arrival rate = 2000 CTx/sec. (b) Committee size $|S^C| = 5$.

Fig. 7. Evaluating system throughput vs. ACP's DAS delay.

A sidechain scheme that uses committee voting for threshold signatures. (iii) *ChainMaker* [19]. Uses cross-chain proxy, SPV, and transaction contracts, supporting cross-chain operations with Fabric. (iv) *WeCross*.[4] Relay-based cross-chain routing that enables data interoperability between Fabric and FISCO BCOS [13]. (v) *BitXHub* [14]. Provides an InterBlockchain Transfer Protocol (IBTP) based on relay chains for reliable routing between permissioned blockchains.

B. Settings

Testbed: It consists of 16 virtual machines, each equipped with an Intel i7-13700 CPU, 8 GB of RAM, and a 5 Mbps network link. To simulate semi-synchronous/asynchronous communication, we use the `tc`[5] command on each machine to control transmission delay Δ (100 $\sim$ 500 ms) and network jitter ($\pm$ 25 ms), and to generate CTx timeouts (i.e., $\Delta > \Delta_{async}$) with a ratio of f. We launch 20 nodes on each machine using Docker containers. We run Hyperledger Fabric v2.4 on half of the machines and ChainMaker v2.1 on the other half to simulate cross-domain data exchanges using heterogeneous permissioned blockchains by different domains. Additionally, the versions of the compared projects are WeCross v1.2 and BitXHub v1.11. For fairness, all baselines are evaluated under the same experimental testbed.

Dataset: We use the real-world IoT datasets `TON_IoT`[6] and `Edge-IIoT`,[7] which comprise millions of raw telemetry data collected from various sensors, including weather, sound, water level, and Modbus sensors [36], [37]. Blockchains deployed by different organizations initially store different IoT data and then exchange a number of data per epoch.

Metrics: The following metrics are used to evaluate the performance of cross-chain interactions. (i) *Throughput*. The number of successful executions of CTx per second by the system (TPS). (ii) *Latency*. The duration for a CTx to stabilize on SC from the time it is initiated by MC. (iii) *Success ratio*. The percentage of successful executions in initiated CTxs (TSR). (iv) *CPU and memory utilization*. The resource consumption of the system during execution. (v) *I/O overhead*. The rate of I/O operations incurred during system execution.

[4]https://github.com/WeBankBlockchain/WeCross
[5]https://linux.die.net/man/8/tc
[6]https://research.unsw.edu.au/projects/toniot-datasets
[7]https://www.kaggle.com/datasets/mohamedamineferrag/edgeiiotset-cybersecurity-dataset-of-iot-iiot

C. Performance Comparison

We perform comprehensive CTx-driven simulations by adjusting the test parameters. Key shared parameters for the baselines include: the number of CTx per test ranges from 1,000 to 10,000. We control Δ to follow a normal distribution between 100 and 500 ms to simulate the typical semi-synchronous communication between IoT devices. At the same time, we control some of the CTxs to timeout (we set $\Delta_{async} = 10$ s) proportionally from 10% to 50% to simulate asynchronous communication. We set the epoch duration to 600 ms. The block generation time for each blockchain is 100 ms. A block is considered stabilized if it is confirmed by the next five blocks (i.e., $T_{Stab} = 500$ ms). Primary configuration distinctions include: the baselines do not employ committee batch proofs ($\delta = 0$ ms); absence of asynchronous buffer pool optimizations; and no delay adjustments. The detailed experimental results are shown in Fig. 5.

1) Throughput: We first normalize the throughput results (NTP) to eliminate environmental differences between the different schemes and facilitate comparisons. The value of NTP reflects the efficiency of the system in highly concurrent data transfer. Fig. 5(a) shows the curves of throughput with an increasing number of CTx. Initially, the NTP of all schemes rises with the number of CTxs. The performance bottleneck is approached when the number of CTxs reaches 4,000 to 6,000. However, AC-BaaS shows an average increase in throughput of 1.71 $\sim$ 5.09 $\times$ compared to other schemes. AC-BaaS exhibits significant throughput advantages for three key reasons. First, AC-BaaS improves proof efficiency by batch committing packaged CTxs through ACP. Second, the test simulates asynchronous communication with timed-out CTxs, where traditional synchronous schemes experience more transaction aborts. In contrast, AC-BaaS demonstrates the benefits of asynchronous concurrency. Third, the optimization of cross-chain buffer pool utility enhances the queuing and processing efficiency of CTxs. Additionally, the actual throughput and our improvements for all schemes are shown in Table III.

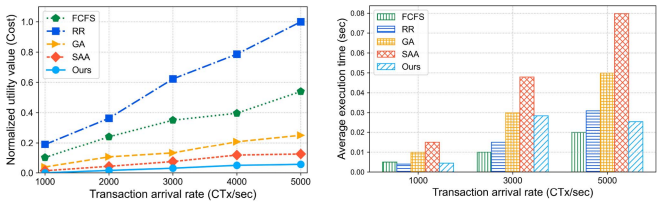
2) Latency: Fig. 5(b) shows the average latency from initiation to stabilization for each CTx at transaction numbers of 2,000, 6,000, and 10,000. The results show that AC-BaaS reduces the latency by 64.36% to 85.49% compared to the other schemes. Notably, the average latency of AC-BaaS is 0.66 s. Although there exists a message delay, AC-BaaS uses ACP to commit an epoch of packed CTxs. Even if part of the transaction times out and retransmits, the asynchronous concurrency greatly reduces the overall transaction latency. AC-BaaS's advantages in asynchronous large-scale data exchange scenarios will be more pronounced. In addition, optimizing the cross-chain buffer pool mitigates the queuing and processing delays associated with CTx consistency, thereby decreasing the overall latency of CTxs.

3) Success Ratio: Fig. 5(c) shows the average success ratio of 10,000 CTxs tested with timeout ratio f set to 10%, 30%, and 50%. The results show that AC-BaaS increases the TSR by 50.26% to 142.74 % compared to the other schemes. Furthermore, the relative improvement of AC-BaaS is more pronounced as f increases. We find that schemes under traditional synchronous settings struggle to handle CTx delays or timeouts properly. Even if they can, transactions are usually aborted due to conflicts. However, AC-BaaS does not require MC to continuously monitor the state of CTxs, and its asynchronous transaction sequence guarantee mechanism can effectively reduce the occurrence of transaction conflicts. In addition, the

TABLE V
PARAMETER SETTINGS AND RESULTS IN THREE SCENARIOS

	Security parameters			Network and load parameters			Δ_{SM} is not introduced (Avg. results)				Δ_{SM} is introduced (Avg. results)			
	Num_{BC}	$\|S^C\|$	Td_{Sig}	Net_{Del}	Net_{Jit}	R_{tx}	TPS	Latency	CPU	Mem.	TPS	Latency	CPU	Mem.
Ordinary	5	5	60%	100 ms	± 5 ms	100 Tx/s	1875.67	0.49 s	71.18%	36.31%	1482.57	0.57 s	72.61%	37.04%
Balancing	10	10	80%	300 ms	± 25 ms	200 Tx/s	1578.98	1.18 s	73.25%	38.13%	1259.91	1.54 s	76.18%	39.73%
Sensitive	20	15	100%	500 ms	± 50 ms	300 Tx/s	1459.64	2.56 s	77.46%	42.57%	1162.78	3.13 s	79.73%	44.82%

* Num_{BC} denotes the number of block confirmations; $|S^C|$ represents the number of committee signatures; Td_{Sig} refers to the DAS signature threshold; Net_{Del} denotes network delay; Net_{Jit} refers to network jitter; R_{tx} represents transaction load fluctuations.



(a) Utility optimization. (b) Running time.

Fig. 8. Comparison of various algorithms for buffer pool optimization.

priority sorting of cross-chain buffer pool proactively resolves dependency conflicts among CTxs during the queuing process.

4) CPU and Memory Utilization: Fig. 5(d) shows the main results of the different schemes in terms of resource overhead during data exchange. It can be seen that the systems are almost fully loaded when the number of CTxs reaches 6,000, and thus the performance reaches a bottleneck. However, AC-BaaS reduces the overall CPU and memory utilization by an average of 14.82% to 25.26% and 45.41% to 54.7%, respectively. This is due to the fact that AC-BaaS uses ACP for batch CTx commitment and reduces the overhead of monitoring and synchronizing states between blockchains. In addition, optimizing the utility of cross-chain buffer pools indirectly decreases the total resource consumption of CTxs.

5) I/O Overhead: To evaluate resource efficiency gains from eliminating continuous state monitoring in asynchronous cross-chain scenarios, we measured system I/O overhead (Fig. 6). Fig. 6(a) shows that I/O operations per CTx increase near-linearly with transaction volume due to storage I/O (e.g., pool read/write, ledger recording) and network I/O (e.g., proof transmission, retransmissions). AC-BaaS reduces per-CTx I/O by over 33.75% via batch ACP commitments and multi-level buffer pools, with greatest gains under high load, avoiding synchronous monitoring bottlenecks and enhancing efficiency in IoT multi-domain settings. Fig. 6(b) demonstrates linear I/O growth per epoch with node count, typical of distributed systems due to compounded computation and communication. AC-BaaS cuts total overhead by over 30.06% versus baselines, confirming its suitability for large-scale multi-domain data exchange with low overall system cost.

D. Evaluation of ACP

1) Theoretical and Experimental Evaluation of Cross-Chain Proofs: Table IV provides a theoretical comparison between AC-BaaS and some SOTA sidechain constructions. AC-BaaS is based on the DAS realization of generating ACPs for n CTxs within an epoch. The cross-chain proofs are aggregated from the committee's signatures. In addition, delayed proofs are implemented in DAS using IVC iterative computations. For any sub-exponential length computation, the complexity in terms of proof size and verification cost is limited by $\mathrm{poly}(\lambda)$. This part of the proof size and computational cost is negligible [26]. Its size is not affected by the chain length or the common prefix. Thus, the proof size complexity of AC-BaaS is $\mathcal{O}(1)$. The computational cost shared by a single CTx is $\mathcal{O}(\frac{|S^C|}{n})$. AC-BaaS is more lightweight than the SOTA constructions.

We measure the generation and verification times for cross-chain proofs. The results in Table IV show that the average generation and verification times of AC-BaaS are 512.3 ms and 83.6 ms, respectively. Since AC-BaaS needs to generate only one cross-chain proof for the CTxs of an epoch, which can be verified quickly, it reduces the proof generation and verification times by 1 to 4 and 1 to 3 orders of magnitude, respectively, compared to SOTA schemes.

2) Performance Evaluation of DAS Dynamic Settings: We vary the size of $|S^C|$ and n to evaluate the system throughput versus DAS delay of ACP, as shown in Fig. 7. We vary the DAS delay within the range $\delta = 400 \sim 600$ ms. Since Δ is a minimum of 100 ms and $T_{Stab} = 500$ ms, δ ensures CTx stability. The results show that throughput increases and then decreases with the increase of δ. This is because the size of δ affects the number of CTx arriving at an epoch, which in turn affects the adequacy of ACP aggregation. Fig. 7(a) shows the results of varying $|S^C|$ for a fixed CTx arrival rate (AR) of 2,000 per second. It can be seen that the larger $|S^C|$ is, the lower the throughput, but its effect is not significant. This is because the larger $|S^C|$, the more committee signatures need to be aggregated, and the aggregation process is rapid. In addition, we control the number of CTx per epoch (i.e., n) by varying the AR. Fig. 7(b) shows the effect of different ARs on throughput. An AR that is too small or too large results in too few or too many CTxs being processed at each epoch, potentially causing the system to be underloaded or overloaded, thereby affecting system throughput.

3) Trade-Off Effects of DAS Delay Settings: To evaluate the impact of DAS latency settings, we model different security application scenarios by adjusting system parameters, thereby assessing the effectiveness of DAS delay settings on the trade-off between security level and system performance. We present a parameterized table (see Table V, left) that outlines the parameter settings for each scenario. As the scenarios transition from *ordinary* to *balancing* to *sensitive*, security requirements progressively intensify, network conditions degrade correspondingly, and transaction load becomes increasingly volatile.

During cross-chain transactions, network conditions and transaction loads vary continuously, necessitating dynamic adjustments to the security margin (Δ_{SM}) imposed by the DAS latency configuration. To streamline system operations, we set the observation window (t_{obs}) to one epoch, i.e., 600 ms. The system periodically calculates the optimal value of Δ_{SM}. The values

TABLE VI
IMPACT OF BUFFER POOL OPTIONS

Metrics	Queue size (# of CTx)			Window size (# of CTx)			Heap priority		
Options	1000	**2000**	3000	200	**400**	600	Time.	Imp.	**Dep.**
TSR (%)	91.43	**92.94**	90.85	91.82	**93.36**	91.36	92.72	91.32	**94.68**

of Δ_{SM} in the previously defined *ordinary*, *balancing*, and *sensitive* scenarios are 100.18 ms, 400.54 ms, and 1200.92 ms, respectively.

The results indicate a consistent performance degradation across all scenarios, with throughput decreasing by approximately 20%. Notably, this reduction remains relatively stable regardless of increasing security requirements, suggesting that the impact of Δ_{SM} on throughput is largely linear and predictable. The *ordinary* scenario exhibits the greatest absolute drop (approximately 393 TPS) due to its higher baseline performance, while the *sensitive* scenario, despite a lower baseline, experiences a similar relative decline. Latency increases show a near-linear correlation with the configured Δ_{SM} . Specifically, the addition of 100.18 ms, 400.54 ms, and 1200.92 ms of Δ_{SM} results in latency increases of approximately 80 ms, 360 ms, and 570 ms in the *ordinary*, *balancing*, and *sensitive* scenarios, respectively. These findings suggest that the majority of the security margin is directly utilized for delay modulation, with minimal computational inefficiency. Moreover, the system demonstrates a robust capacity to adapt to varying network conditions and transactional dynamics, effectively tracking the configured Δ_{SM} and maintaining precise control over cross-chain latency.

In terms of resource overhead, increasing security levels and volatility in network or transaction load lead to modest rises in computational cost. CPU utilization increases by 1.4% to 2.9%, while memory usage grows by 0.7% to 2.3%. The *balancing* scenario incurs the highest CPU overhead, likely due to more frequent dynamic adjustments triggered by the balanced contributions of R_{net} and R_{tx}. In contrast, although the *sensitive* scenario applies the largest Δ_{SM}, its impact primarily translates into passive wait time rather than additional computation, resulting in comparatively lower resource consumption.

Therefore, the security margin Δ_{SM} in DAS effectively balances cross-chain security with performance & overhead, introducing predictable and controllable impacts on throughput, latency, and resource consumption. By flexibly selecting or dynamically switching among the *ordinary*, *balancing*, and *sensitive* modes based on the security level and real-time requirements of specific application scenarios, the system achieves an optimal trade-off between security and efficiency. In addition, we discuss typical use cases of AC-BaaS in different scenarios in Appendix E, available in the online.

E. Evaluation of CTx Buffer Pool

1) Impact of Buffer Pool Metric Options: To verify the effectiveness of the transaction sequence guarantee mechanism in reducing conflicts, we evaluate the impact of different metric options for the buffer pool on the transaction success ratio. We attach random weights and 20% dependencies to the transactions. The timeout rate $f = 30\%$. Table VI presents the average results of our multiple control variable tests. Each metric displays three key options. The results indicate that moderately increasing the queue size can improve TSR, but an excessively large queue may reduce processing efficiency. A larger window helps improve TSR, but an excessively large window may increase the probability of conflicts. The dependency prioritization strategy improves TSR more than timestamp and importance prioritization, indicating that considering inter-transaction dependencies in multilevel buffer pools is effective.

2) Buffer Pool Utility Optimization Evaluation: To validate the effectiveness of the utility optimization algorithm for the buffer pool mechanism, we compare the proposed heuristic optimization algorithm with existing classical algorithms. Specifically, we selected the first-come first-served (FCFS) and round robin (RR) algorithms as benchmark methods, while the genetic algorithm (GA) and simulated annealing algorithm (SAA) serve as efficient heuristic optimization techniques. To assess the performance of the algorithms under varying load conditions, we set the transaction arrival rate to range from 1,000 to 5,000, simulating the utility and convergence of each algorithm across diverse buffer pool load scenarios.

Fig. 8 presents the results from multiple simulations. Overall, the proposed heuristic algorithm demonstrates a substantial utility improvement over the existing classical algorithms, with `Cost` reductions ranging from approximately 59.2% to 94.9%. Furthermore, the proposed algorithm also demonstrates considerable efficiency in terms of runtime.

Specifically, Fig. 8(a) illustrates that, compared to the benchmark algorithms (e.g., FCFS and RR), our algorithm achieves an average improvement of over 90% in optimization effectiveness, with the improvement becoming more pronounced as the transaction load increases. This is attributed to the fact that the benchmark algorithms lack flexible resource allocation strategies and priority scheduling when processing transactions, which results in lower resource utilization and, consequently, higher utility values. In high-load scenarios, both transaction processing delays and conflict rates increase significantly. In contrast, our heuristic algorithm optimizes resource allocation in real-time by adaptively adjusting the parameters, significantly reducing system latency and conflict rates, thereby effectively lowering the utility value. Compared to other heuristic optimization algorithms (e.g., GA and SAA), our algorithm also achieves an average improvement of 72.5%. This difference arises because our algorithm employs a lighter heuristic tuning strategy for parameter updating, enabling it to find the near-optimal solution more efficiently than the complex global search processes of GA and SAA, thereby significantly reducing computational overhead while ensuring utility optimization.

Fig. 8(b) shows that FCFS and RR take on average 40.1% and 14.5% less execution time than our algorithm, respectively. However, although FCFS and RR can perform quickly in the simple case, the lack of optimization leads to their poorer utility values. Compared to other heuristic algorithms, our algorithm reduces the 35.0% to 59.1%. This result demonstrates that, in addition to utility optimization, our heuristic algorithm not only maintains high performance but also significantly reduces computational complexity and avoids the high time overhead typically associated with traditional heuristics. The reason for this difference is that the GA and SAA algorithms GA and SAA algorithms, on the other hand, rely on global searches and multiple iterations to find the optimal solution, resulting in higher computational overhead. Our algorithm quickly converges to

a more optimal solution in a shorter period through adaptive parameter tuning and micro-batching local optimization strategies, with computational overhead remaining nearly linear as transaction volume increases.

Overall, our heuristic algorithm demonstrates efficient runtime performance while optimizing utility, particularly in high-load transaction environments. Our algorithm strikes an effective balance between performance and resource consumption, successfully addressing complex asynchronous cross-chain transaction scheduling and optimization problems.

VIII. Conclusion and Future Work

To facilitate the trusted exchange of IoT multi-domain data, we propose AC-BaaS, an asynchronous cross-blockchain as a service framework. To provide asynchronous cross-chain proofs (ACPs) for the validity of cross-chain transactions, we introduce a cryptographic primitive, delayed aggregate signature (DAS), and explore the trade-offs involved in setting the DAS delay size. To ensure the consistency of asynchronous cross-chain transactions, we design a transaction sequence guarantee mechanism and optimize its utility. Our analysis demonstrates that AC-BaaS is secure. Prototype evaluation results show that AC-BaaS outperforms existing SOTA schemes. Our future work will focus on designing a fair incentive mechanism for AC-BaaS, aiming to encourage active participation from AsyncSC members in providing AC-BaaSs. This is expected to enhance the liquidity and quality of cross-domain data exchange.

References

[1] L. Yang et al., "AsyncSC: An asynchronous sidechain for multi-domain data exchange in Internet of Things," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2025, pp. 1–10.

[2] S. He, K. Shi, C. Liu, B. Guo, J. Chen, and Z. Shi, "Collaborative sensing in Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1435–1474, Third Quarter 2022.

[3] Cisco, "Powering an inclusive, digital future for all," 2023. [Online]. Available: https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2023/m01/powering-an-inclusive-digital-future-for-all.html

[4] X. Chen, Q. Cheng, X. Chen, and X. Luo, "A blockchain-based cross-domain data transmission scheme for industrial Internet of Things with edge-cloud computing," *IEEE Trans. Serv. Comput.*, vol. 18, no. 5, pp. 2489–2502, Sep./Oct. 2025.

[5] X. Zhu, J. Zheng, B. Ren, X. Dong, and Y. Shen, "MicrothingsChain: Blockchain-based controlled data sharing platform in multi-domain IoT," *J. Netw. Netw. Appl.*, vol. 1, no. 1, pp. 19–27, 2021.

[6] W. Tong et al., "TI-BIoV: Traffic information interaction for blockchain-based IoV with trust and incentive," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21528–21543, Dec. 2023.

[7] Y. Feng, W. Zhang, X. Luo, and B. Zhang, "A consortium blockchain-based access control framework with dynamic orderer node selection for 5G-enabled industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 4, pp. 2840–2848, Apr. 2022.

[8] M. Zhang, Q. Qu, L. Ning, and J. Fan, "On time-aware cross-blockchain data migration," *Tsinghua Sci. Technol.*, vol. 29, no. 6, pp. 1810–1820, 2024.

[9] S. A. Back et al., "Enabling blockchain innovations with pegged," Oct. 2014. [Online]. Available: https:// blockstream.com/sidechains.pdf

[10] A. Kiayias and D. Zindros, "Proof-of-work sidechains," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, Springer, 2019, pp. 21–34.

[11] P. Gaži, A. Kiayias, and D. Zindros, "Proof-of-stake sidechains," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 139–156.

[12] L. Yin, J. Xu, and Q. Tang, "Sidechains with fast cross-chain transfers," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3925–3940, Nov./Dec. 2021.

[13] "FISCO BCOS," 2024. [Online]. Available: http://www.fisco-bcos.org/

[14] S. Ye et al., "BitXHub: Side-relay chain based heterogeneous blockchain interoperable platform," *Comput. Sci.*, vol. 47, no. 6, pp. 300–308, 2020.

[15] L. Yin, J. Xu, K. Liang, and Z. Zhang, "Sidechains with optimally succinct proof," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3375–3389, Jul./Aug. 2024.

[16] T. Xie et al., "zkBridge: Trustless cross-chain bridges made practical," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 3003–3017.

[17] J. Singh and J. D. Michels, "Blockchain as a service (BaaS): Providers and trust," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops*, 2018, pp. 67–74.

[18] E. Androulaki et al., "HyperLedger Fabric: A distributed operating system for permissioned blockchains," in *Proc. 13th EuroSys Conf.*, 2018, pp. 1–15.

[19] "ChainMaker," 2024. [Online]. Available: https://chainmaker.org.cn/

[20] S. D. Lerner et al., "RSK: A bitcoin sidechain with stateful smart-contracts," May 2022. [Online]. Available: https://eprint.iacr.org/2022/684.pdf

[21] J. Kwon and E. Buchman, "Cosmos Whitepaper," Dec. 2020. [Online]. Available: https://wikibitimg.fx994.com/attach/2020/12/16623142020/WBE16623142020_55300.pdf

[22] T. Xie, K. Gai, L. Zhu, S. Wang, and Z. Zhang, "RAC-Chain: An asynchronous consensus-based cross-chain approach to scalable blockchain for metaverse," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 20, no. 7, 2024, Art. no. 187.

[23] H. Huang et al., "Scheduling most valuable committees for the sharded blockchain," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 3284–3299, Dec. 2023.

[24] M. Zhai et al., "Secret multiple leaders & committee election with application to sharding blockchain," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5060–5074, 2024.

[25] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Springer, 2001, pp. 514–532.

[26] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, "Verifiable delay functions," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2018, pp. 757–788.

[27] Y. Zhao, "Practical aggregate signature from general elliptic curves, and applications to blockchain," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2019, pp. 529–538.

[28] J. Nick, T. Ruffing, and Y. Seurin, "MuSig2: Simple two-round Schnorr multi-signatures," in *Proc. Annu. Int. Cryptol. Conf.*, Springer, 2021, pp. 189–221.

[29] B. Wesolowski, "Efficient verifiable delay functions," *J. Cryptol.*, vol. 33, pp. 2113–2147, 2020.

[30] A. Boldyreva, "Efficient threshold signatures, multisignature and blind signature schemes based on the gap-diffie-hellman-group signature scheme," in *Proc. Int. Workshop Theory Pract. Public Key Cryptogr.*, 2002, pp. 31–46.

[31] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," *Commun. ACM*, vol. 59, no. 2, pp. 103–112, 2016.

[32] A. Warner and T. F. Keefe, "Version pool management in a multilevel secure multiversion transaction manager," in *Proc. IEEE Symp. Secur. Privacy*, 1995, pp. 169–182.

[33] M. Saad, L. Njilla, C. Kamhoua, J. Kim, D. Nyang, and A. Mohaisen, "Mempool optimization for defending against DDoS attacks in PoW-based blockchain systems," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency*, 2019, pp. 285–292.

[34] A. Zamyatin et al., "SoK: Communication across distributed ledgers," in *Proc. 25th Int. Conf. Financial Cryptogr. Data Secur.*, Springer, 2021, pp. 3–36.

[35] J. Chen, J. Teh, Z. Liu, C. Su, A. Samsudin, and Y. Xiang, "Towards accurate statistical analysis of security margins: New searching strategies for differential attacks," *IEEE Trans. Comput.*, vol. 66, no. 10, pp. 1763–1777, Oct. 2017.

[36] T. M. Booij et al., "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 485–496, Jan. 2022.

[37] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

Lingxiao Yang (Member, IEEE) received the BE and PhD degrees from the School of Computer Science and Technology, Xidian University, Xi'an, China, in 2018 and 2025, respectively. He is currently a lecturer with the School of Artificial Intelligence and Computer Science, Shaanxi Normal University. He is also with the Engineering Research Center of Blockchain Technology Application and Evaluation. His research interests interests include blockchain and smart system security.

Xuewen Dong (Member, IEEE) received the BE, MS, and PhD degrees in computer science and technology from Xidian University, Xi'an, China, in 2003, 2006, and 2011, respectively. From 2016 to 2017, he was with Oklahoma State University, Stillwater, OK, USA, as a visiting scholar. He is currently a professor with the School of Computer Science and Technology, Xidian University. His research interests include blockchain and smart system security.

Zhiguo Wan (Senior Member, IEEE) is currently a principal investigator with Zhejiang Laboratory, Hangzhou, Zhejiang, China. He authored or coauthored more than 100 academic papers, including those in top international conferences and journals such as ACM CCS, INFOCOM, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, and *IEEE Transactions on Parallel and Distributed Systems*. His works have been cited over 5,000 times according to Google Scholar, with a single paper cited more than 700 times. His research interests mainly include security and privacy for distributed systems, such as cloud computing, Internet of Things, and blockchain. He has led four projects funded by the National Natural Science Foundation of China (NSFC) and the Major Basic Research Program of the Shandong Provincial Natural Science Foundation. He was recipient of the Second Prize of the Zhejiang Provincial Science and Technology Progress Award. He is also a senior member of the China Computer Federation (CCF).

Sheng Gao (Member, IEEE) received the BS degree in information and computation science from the Xi'an University of Posts and Telecommunications, in 2009, and the PhD degree in computer science and technology from Xidian University, in 2014. He is currently a professor with the School of Information, Central University of Finance and Economics. He has authored or coauthored more than 30 articles in refereed international journals and conferences. His research interests include data security, privacy computing, and blockchain technology.

Wei Tong (Member, IEEE) received the BS degree in Internet of Things from the Jiangsu University of Technology, Zhenjiang, China, in 2017, and the PhD degree in cyberspace security from Xidian University, Xi'an, China, in 2022. He is currently a lecturer with the School of Information Science and Engineering, Zhejiang Sci-Tech University. His research interests include network security and data security, especially blockchain technology & application, and trustworthy distributed AI.

Yong Yu (Fellow, IEEE) received the PhD degree in cryptography from Xidian University, in 2008. He is currently a professor with Shaanxi Normal University, China. He also holds the Prestigious One Hundred Talent professorship of Shaanxi Province. He has authored more than 100 referred journal articles and conference papers. His research interests include blockchain and cloud security. He is an associate editor for *IEEE Transactions on Dependable and Secure Computing*.

Yulong Shen (Senior Member, IEEE) received the BS and MS degrees in computer science and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2002, 2005, and 2008, respectively. He is currently a professor with the School of Computer Science and Technology, Xidian University, where he is also the associate director with the Shaanxi Key Laboratory of Network and System Security and a member with the State Key Laboratory of Integrated Services Networks. His research interests include wireless network security and cloud computing security. He was on the Technical Program Committees of several international conferences, including ICEBE, INCoS, CIS, and SOWN.